

CSE 344 - System Programming

Homework #3

Due Date: 15/04/2026 23:59 PM

- Do not hesitate, and feel free to ask any questions about homework.
- Primarily, ask your questions below the assignment as a reply on Teams so that everyone can benefit from them.

Multi-Process Word Transportation and Concurrent Sorting System

1. Overview

In this homework, you will design and implement a **multi-process system** that reads words from a text file, distributes them across floors, transports their characters between floors, and reconstructs them using concurrent sorting processes.

The goal of this assignment is not simply moving data from one place to another. Instead, the main objective is:

How can multiple independent processes operate concurrently on shared data structures while maintaining correctness, consistency, and synchronization?

This system includes:

- multiple processes working simultaneously,
- multiple active words at the same time,
- multiple character transfers occurring concurrently,
- multiple sorting processes operating on the same floor,
- shared data structures accessed concurrently,
- floor capacity constraints,
- two independent elevator subsystems,
- and a coordinated system-wide termination mechanism.

This assignment directly involves:

- process creation
- inter-process communication
- shared memory
- synchronization
- race conditions
- starvation, waiting, and deadlock risks
- controlled system termination

The design is also intentionally suitable for a **thread-based extension** in future work.

2. System Scenario

Imagine a building with multiple floors.

You are given a .txt file containing multiple words. Each word represents a “product” that must be processed by the system.

Each word:

- has a unique **ID**
- has a designated **sorting floor**
- is initially placed on an **arrival floor**
- and must eventually be reconstructed on its sorting floor

Words are not transferred as a whole. Instead:

- each word is decomposed into **characters**
- characters are transported **one by one**
- characters arrive in **random order**
- sorting processes reconstruct the word based on its original structure

Example:

apple

Characters may arrive as:

p, e, a, l, p

Sorting processes must rearrange them correctly.

3. Input File Format

The file should not include any blank, empty or white-space line. Each line in the input file must contain:

- word_id
- word
- sorting_floor

They are separated by only a single space. After `sorting_floor`, there should be a newline immediately.

Example:

```
101 apple 7
```

```
102 process 3
```

```
103 system 12
```

```
104 kernel 5
```

```
105 thread 9
```

All words should be in English, which means words in input file should include only English alphabet letters.

All words should be lowercase.

Each character derived from a word must carry:

- word_id
- character
- original_index

Example for "apple":

- (101, 'a', 0)
- (101, 'p', 1)
- (101, 'p', 2)
- (101, 'l', 3)
- (101, 'e', 4)

This metadata is essential for correct reconstruction.

4. Process Types in the System

4.1 Parent / Coordinator Process

Responsible for:

- reading the input file
- initializing shared memory
- creating synchronization primitives
- spawning child processes
- monitoring system completion
- terminating all processes
- printing final statistics
- All child processes in the system must be created exclusively by the parent process. No other process is allowed to create system-wide processes
- The parent process is responsible for generating the final output text file after the system completes

4.2 Word-Carrier Processes

Responsible for admitting words into the system.

Behaviour:

- Once created word-carrier processes never change its floor and it can bring word from file to only its floor where it is created. This is called arrival floor.
- scan the input list using **round-robin**
- select an unclaimed word
- check capacity of:
 - arrival floor
 - sorting floor (from input)
- if both are available → activate the word and put it to the arrival floor to start transportation operation
- otherwise → release the word for future attempts

This is an **all-or-nothing admission control**.

4.3 Letter-Carrier Processes

Responsible for transporting characters.

Behaviour:

- select an **available, unclaimed** character task from the current floor
- if destination differs → use **delivery elevator**
- if same floor → place directly
- after delivery → stay on destination floor
- search for new work locally
- if idle → use **reposition elevator**
- a letter-carrier process can move away from its initial floor during execution.

These processes are dynamic and mobile.

If there is no letter-carrier process on any floor at any time, the parent process should immediately create the number of new letter-carrier processes, initially specified as an argument on the terminal for letter-carrier processes, on that floor.

A letter-carrier must never process the same character task more than once, and no two carriers may claim the same task.

Remember that the two things below should be done concurrently in arrival floors;

- word-carrier processes should bring the words to the arrival floors,
- letter-carrier processes should pick a word randomly from the arrival floors, then randomly get a letter of that word and start to transport to the sorting floors.

4.4 Sorting Processes

Operate on sorting floors.

- multiple sorting processes per floor are allowed
- but:

Any sorting process in a floor can operate on any word in that floor.

Only ONE sorting process may operate on a specific word at a time.

Responsibilities:

- scan sorting area
- fix correctly placed characters
- move or swap incorrect ones
- mark completion
- never change its initial sorting floor

Remember that the two things below should be done concurrently in sorting floors;

- Characters of words should be randomly transported to the related area allocated for the word that they belong to in sorting floors and placed in the first empty place where letter-carrier processes find,
- sorting processes should sort those characters based on the original word, even though not all letters of any word have been transported yet.

4.5 Delivery Elevator Process

Handles character transportation.

Rules:

- direction-based movement (UP / DOWN)
- serves requests in current direction first
- compulsory reverses at top/bottom floors (it has to change its direction UP->DOWN or vice versa)
- the elevator continues to work in the current direction until these two conditions are satisfied;
 - there is no received request from the floors that are located above or below the current floor, based on its current direction
 - the elevator contains no characters to transport(idle)
- idle → adopts direction of first request

Remember that the delivery elevator has a capacity indicating the maximum number of characters transported at any time.

4.6 Reposition Elevator Process

Handles idle letter-carrier relocation.

- separate from delivery elevator but same working policy as in the delivery elevator
- used only when carriers are idle
- moves carriers to random floors

Remember that the reposition elevator has a capacity indicating the maximum number of letter-carrier processes transported at any time.

4.7 Process Initialization

At system startup, the parent process must create multiple processes per floor.

- multiple word-carrier processes per floor
- multiple letter-carrier processes per floor
- multiple sorting processes per floor
- one delivery elevator
- one reposition elevator

Each letter-carrier process starts execution on an assigned initial floor but may later move to other floors during execution.

5. Round-Robin Word Selection

Word-carrier processes must scan the input list in **round-robin order**.

This ensures:

- fairness
- reduced starvation
- predictable behavior

Important:

- scanning must be synchronized
- word claiming must be atomic

6. Word Admission Policy

A word is admitted ONLY if:

- arrival floor has capacity
- sorting floor has capacity

If either fails:

- the word is released
- may be selected again later

The capacity checks for arrival floor and sorting floor must be performed atomically to prevent inconsistent admission states.

Pay attention that on the same floor, these two types of words can be located at the same time;

- words taken from the input text file and should be transported to the different floors,
- words whose arrival floor is different, but they should be sorted on that floor.

7. Floors

Each word has:

Arrival Floor

- Depending on which floor's word-carrier process claim the word
- initial placement

Sorting Floor

- defined in input
- final reconstruction location

Flow:

1. word is admitted
2. placed on arrival floor
3. characters extracted
4. characters transported
5. sorted on sorting floor

8. Floor Capacity

Each floor has limited capacity:

- maximum number of active words
- contains currently being sorted and transported words

If full:

- new words cannot be admitted
- word taken by txt should be released for future attempts

9. Character Transportation

Each character is a task:

- must be claimed exactly once
- must be delivered exactly once

Characters are placed into sorting area:

- NOT directly to final position
- first empty, non-fixed slot

10. Sorting Data Structure

Each word has:

- original word
- `sorting_area[]`
- `occupied[]`
- `fixed[]`
- claimed flag
- admitted flag
- completed flag

Set all flags to 0 initially.

If a word-carrier process picks a word to bring to the arrival floor, it sets the claimed flag to 1.

If a word-carrier process finds an available place in the arrival floor for the word that it brings, then the word-carrier process sets the admitted flag to 1. Otherwise, it leaves the admitted flag as 0 and sets the claimed flag to 0 again.

If all indexes of the `fixed` are set to 1, then the sorting process sets the completed flag to 1.

11. Sorting Rules

For each slot:

Case 1: Empty

- do nothing

Case 2: Fixed

- do nothing

Case 3: Correct position

- mark as fixed

Case 4: Incorrect position

- find correct target

If:

- target empty → move
- target occupied & not fixed → swap
- target fixed → do nothing

12. Fixed Positions

If `fixed[i] == 1`:

- cannot be modified
- cannot be swapped
- cannot receive new characters

13. Multiple Sorting Processes

Allowed per floor.

Constraint:

Only one process per word at a time.

Requires per-word locking.

14. Letter-Carrier Behaviour

After delivery:

- stays on destination floor
- does NOT return
- selects next task locally
- may reposition if idle

15. Idle Behaviour & Repositioning

If no task available:

- request reposition
- use the reposition-elevator to move to random floor
- resume work

16. Elevators

Delivery Elevator

- handles characters
- follows direction rules

Reposition Elevator

- handles idle carriers
- independent queue

17. Termination Condition

The program terminates ONLY when these three conditions are satisfied;

- all words are sorted in their specified sorting floor, which means all completed flags are set to 1,
- the final output file is generated,
- summary information is printed on the terminal.

Parent process handles shutdown.

18. Summary

Program should report on terminal:

- total words
- completed words
- retries
- characters transported
- per-process statistics
- elevator usage

19. Final Output File

The file should not include any blank, empty or white-space line. Each line in the output file must contain:

`<word_id> <word> <sorting_floor>`

They are separated by only a single space. After `sorting_floor`, there should be a newline immediately. Also;

- must include ALL words
- must use **original sorting floor (from input)**
- must be sorted by:
 1. `sorting_floor`
 2. `word_id`

```
102 process 3
104 kernel 5
101 apple 7
105 thread 9
103 system 12
```

The arrival floor must NOT affect the final output.

20. Runtime Parameters

The program must accept configuration parameters from the command line.

These parameters define the system size, process counts, and capacity constraints.

Students are expected to use standard command-line parsing mechanisms (e.g., `getopt`) to process these arguments. Students are encouraged to use `getopt()` for argument parsing.

20.1 Mandatory Parameters

The following parameters must be provided by the user:

Number of Floors

-f <num_floors>

- Total number of floors in the system.
- Floors are assumed to be numbered from 0 to num_floors - 1.

Word-Carrier Processes per Floor

-w <word_carriers_per_floor>

- Number of word-carrier processes initially created for each floor.

Letter-Carrier Processes per Floor

-l <letter_carriers_per_floor>

- Number of letter-carrier processes initially created for each floor.

Sorting Processes per Floor

-s <sorting_processes_per_floor>

- Number of sorting processes created for each floor.

Floor Capacity (Active Words per Floor)

-c <max_words_per_floor>

- Maximum number of active words allowed on any floor.
- Used in admission control.

Delivery Elevator Capacity

-d <delivery_elevator_capacity>

- Maximum number of characters allowed to be inside the delivery elevator.

Reposition Elevator Capacity

-r <reposition_elevator_capacity>

- Maximum number of letter-carrier processes allowed to be inside the reposition elevator.

Input File

-i <input_file>

- Path to the input .txt file.

Output File

-o <output_file>

- Path to the output .txt file to be generated at the end.

20.2 Parameter Constraints

The program must validate all input parameters before execution.

The following constraints must be enforced:

- num_floors >= 1
- word_carriers_per_floor >= 1
- letter_carriers_per_floor >= 1
- sorting_processes_per_floor >= 1
- max_words_per_floor >= 1
- elevator capacities must be positive integers
- input file must exist and be readable

Invalid parameters must result in:

- a clear error message printed to stderr
- program termination with non-zero exit code

20.3 Example Usage

```
./hw3 -f 10 -w 2 -l 3 -s 2 -c 5 -d 8 -r 5 -i input.txt -o output.txt
```

Program is starting...

Input file is being read...

Shared memory is initialized...

Synchronization primitives are created...

Processes are being created...

[PID:3000] Parent process started

--- Initializing Floor 0 ---

[PID:3001] Word-carrier-process_0 initialized on floor 0

[PID:3002] Word-carrier-process_1 initialized on floor 0

[PID:3003] Letter-carrier-process_0 initialized on floor 0

[PID:3004] Letter-carrier-process_1 initialized on floor 0

[PID:3005] Sorting-process_0 initialized on floor 0

[PID:3006] Sorting-process_1 initialized on floor 0

--- Initializing Floor 1 ---

[PID:3007] Word-carrier-process_2 initialized on floor 1
[PID:3008] Word-carrier-process_3 initialized on floor 1
[PID:3009] Letter-carrier-process_2 initialized on floor 1
[PID:3010] Letter-carrier-process_3 initialized on floor 1
[PID:3011] Sorting-process_2 initialized on floor 1
[PID:3012] Sorting-process_3 initialized on floor 1

--- Initializing Floor 2 ---

[PID:3013] Word-carrier-process_4 initialized on floor 2
[PID:3014] Word-carrier-process_5 initialized on floor 2
[PID:3015] Letter-carrier-process_4 initialized on floor 2
[PID:3016] Letter-carrier-process_5 initialized on floor 2
[PID:3017] Sorting-process_4 initialized on floor 2
[PID:3018] Sorting-process_5 initialized on floor 2

[PID:3019] Delivery elevator process started
[PID:3020] Reposition elevator process started

[PID:3001] Word-carrier-process_0 claimed word 101
[PID:3001] Word 101 admitted to floor 0 (sorting floor: 2)

[PID:3007] Word-carrier-process_2 claimed word 102
[PID:3007] Word 102 admitted to floor 1 (sorting floor: 0)

[PID:3013] Word-carrier-process_4 claimed word 103
[PID:3013] Word 103 admitted to floor 2 (sorting floor: 1)

[PID:3003] Letter-carrier-process_0 selected char 'a' of word 101 from floor 0
[PID:3003] Letter-carrier-process_0 requested delivery elevator from floor 0 to floor 2

[PID:3009] Letter-carrier-process_2 selected char 'b' of word 102 from floor 1
[PID:3009] Letter-carrier-process_2 requested delivery elevator from floor 1 to floor 0

[PID:3015] Letter-carrier-process_4 selected char 'd' of word 103 from floor 2
[PID:3015] Letter-carrier-process_4 requested delivery elevator from floor 2 to floor 1

[PID:3019] Delivery elevator arrived at floor 0
[PID:3019] Delivery elevator pick up (currently 1 letter-carrier inside):
Letter-carrier-process_0 carrying char 'a' of word 101

[PID:3019] Delivery elevator moving UP

[PID:3002] Word-carrier-process_1 claimed word 104
[PID:3002] Word 104 admitted to floor 0 (sorting floor: 1)

[PID:3010] Letter-carrier-process_3 selected char 'o' of word 102 from floor 1
[PID:3010] Destination is same floor? No
[PID:3010] Letter-carrier-process_3 requested delivery elevator from floor 1 to floor 0

[PID:3019] Delivery elevator drop off at floor 2 (currently 0 letter-carrier inside):
Letter-carrier-process_0 carrying char 'a' of word 101

[PID:3003] Letter-carrier-process_0 brought char 'a' of word 101 to floor 2

[PID:3017] Sorting-process_4 detected char 'a' of word 101 on floor 2
[PID:3017] Sorting-process_4 placed char 'a' of word 101 into sorting area
[PID:3017] Sorting-process_4 determined correct position for char 'a'
[PID:3017] Sorting-process_4 fixed char 'a' of word 101 on floor 2

[PID:3014] Word-carrier-process_5 claimed word 105
[PID:3014] Word 105 admitted to floor 2 (sorting floor: 0)

[PID:3004] Letter-carrier-process_1 selected char 'p' of word 101 from floor 0
[PID:3004] Letter-carrier-process_1 requested delivery elevator from floor 0 to floor 2

[PID:3019] Delivery elevator arrived at floor 1
[PID:3019] Delivery elevator pick up (currently 2 letter-carrier inside):
 Letter-carrier-process_2 carrying char 'b' of word 102
 Letter-carrier-process_3 carrying char 'o' of word 102

[PID:3019] Delivery elevator moving DOWN

[PID:3019] Delivery elevator drop off at floor 0 (currently 0 letter-carrier inside):
 Letter-carrier-process_2 carrying char 'b' of word 102
 Letter-carrier-process_3 carrying char 'o' of word 102

[PID:3009] Letter-carrier-process_2 brought char 'b' of word 102 to floor 0
[PID:3010] Letter-carrier-process_3 brought char 'o' of word 102 to floor 0

[PID:3005] Sorting-process_0 detected char 'b' of word 102 on floor 0
[PID:3005] Sorting-process_0 placed char 'b' into sorting area

[PID:3006] Sorting-process_1 detected char 'o' of word 102 on floor 0
[PID:3006] Sorting-process_1 placed char 'o' into sorting area
[PID:3006] Sorting-process_1 swap performed for word 102
[PID:3006] Sorting-process_1 fixed one index of word 102

[PID:3016] Letter-carrier-process_5 found no available task on floor 2
[PID:3016] Letter-carrier-process_5 requested reposition elevator from floor 2

[PID:3020] Reposition elevator arrived at floor 2
[PID:3020] Reposition elevator pick up (currently 1 letter-carrier inside):
 Letter-carrier-process_5

[PID:3020] Reposition elevator moving DOWN

[PID:3008] Word-carrier-process_3 claimed word 106
[PID:3008] Word 106 admitted to floor 1 (sorting floor: 2)

[PID:3011] Sorting-process_2 is scanning word 103 on floor 1
[PID:3011] Sorting-process_2 found char 'd' misplaced
[PID:3011] Sorting-process_2 moved char 'd' to correct index
[PID:3011] Sorting-process_2 fixed char 'd' of word 103 on floor 1

[PID:3019] Delivery elevator arrived at floor 0
[PID:3019] Delivery elevator pick up (currently 1 letter-carrier inside):
 Letter-carrier-process_1 carrying char 'p' of word 101

[PID:3019] Delivery elevator moving UP

[PID:3020] Reposition elevator drop off at floor 0 (currently 0 letter-carrier inside):
Letter-carrier-process_5

[PID:3016] Letter-carrier-process_5 resumed work on floor 0
[PID:3016] Letter-carrier-process_5 selected char 'k' of word 105 from floor 0
[PID:3016] Letter-carrier-process_5 requested delivery elevator from floor 0 to floor 0
[PID:3016] Destination is same floor → direct placement
[PID:3016] Letter-carrier-process_5 brought char 'k' of word 105 to floor 0

[PID:3005] Sorting-process_0 detected char 'k' of word 105 on floor 0
[PID:3005] Sorting-process_0 placed char 'k' into sorting area
[PID:3005] Sorting-process_0 fixed char 'k' of word 105

[PID:3019] Delivery elevator drop off at floor 2 (currently 0 letter-carrier inside):
Letter-carrier-process_1 carrying char 'p' of word 101

[PID:3004] Letter-carrier-process_1 brought char 'p' of word 101 to floor 2

[PID:3018] Sorting-process_5 is scanning word 101 on floor 2
[PID:3018] Sorting-process_5 found char 'p' misplaced
[PID:3018] Sorting-process_5 swapped char 'p' with another non-fixed char
[PID:3018] Sorting-process_5 fixed one more position of word 101

[PID:3017] Word 101 COMPLETED
[PID:3006] Word 102 COMPLETED
[PID:3011] Word 103 COMPLETED
[PID:3012] Word 104 COMPLETED
[PID:3005] Word 105 COMPLETED
[PID:3018] Word 106 COMPLETED

All words have been transported and sorted...
Output file is being created...

System Summary:

Total words: 6
Completed words: 6
Retries: 2
Characters transported: 27
Delivery elevator operations: 14
Reposition elevator operations: 1

Program terminated successfully.

21. Learning Objectives

Students should understand:

- process isolation
- shared memory usage
- synchronization necessity
- atomic operations
- race condition avoidance
- system-wide coordination
- termination detection

22. Important Warnings

Students must ensure:

- no duplicate claims
- no concurrent modification of same word
- correct fixed handling
- no busy waiting
- correct lock ordering
- no premature termination

23. System Flow Summary

1. Parent initializes system
2. Word-carriers admit words
3. Letter-carriers transport characters
4. Sorting processes reconstruct words
5. Completed words are marked
6. System terminates when all work is done

24. Why This Assignment is Challenging

This is not a single problem, but a combination of:

- scheduling
- synchronization
- data consistency
- concurrent modification
- termination detection

25. Report Requirements

Students must submit a **PDF report** explaining their design, implementation, and testing of the system.

The report must be written in **English** and must be **clear, structured, and concise**.

25.1 Mandatory Sections

The report must include the following sections:

Introduction

- Brief description of the problem
- Overview of the system
- Explanation of the main challenges (concurrency, synchronization, process coordination)

System Architecture

- Description of all process types:
 - parent process
 - word-carrier processes
 - letter-carrier processes
 - sorting processes
 - delivery elevator
 - reposition elevator
- Explanation of how processes interact
- Diagram is strongly recommended (optional but encouraged)

Data Structures and Shared Memory Design

- Description of all shared data structures
- Explanation of:
 - word structure (claimed, admitted, completed flags)
 - sorting structures (sorting area, fixed positions, etc.)
 - elevator queues
- How shared memory is organized and accessed

Synchronization Strategy

- Explanation of synchronization mechanisms used (mutexes, semaphores, etc.)
- How race conditions are prevented
- How consistency is maintained
- Explanation of critical sections

Word Admission Logic

- How round-robin selection is implemented
- How atomic capacity checking is ensured
- What happens when admission fails

Character Transportation and Elevators

- How character tasks are generated and assigned
- How delivery elevator works (direction, capacity, requests)
- How reposition elevator works
- How idle letter-carriers are handled

Sorting Algorithm

- Explanation of how characters are placed
- Handling of:
 - correct position detection
 - swapping
 - fixed positions
- Handling of repeated characters

Termination Detection

- Explanation of how the system detects completion
- Use of:
 - claimed / admitted / completed flags
- How the parent decides when to terminate

Output File Generation

- How the final output file is generated
- How sorting by sorting_floor and word_id is ensured
- How correctness of output is verified

Test Case Scenario

Students must include:

- the mandatory test case input file
- the command used to run the program
- the produced output file
- explanation of observed behavior

Challenges and Solutions

- Problems encountered during implementation
- Deadlock or race condition issues (if any)
- How they were resolved

Additional Tests (Optional but Recommended)

- Any additional test cases used
- Stress tests (e.g., high number of processes, small capacities)
- Observations from these tests

25.2 Important Requirements

- The report must clearly explain **design decisions**, not just describe code.
- Simply pasting code or logs without explanation is not sufficient.
- The report must reflect the student's understanding of:
 - concurrency
 - synchronization
 - process coordination

25.3 Formatting Rules

- File format: PDF
- Language: English
- Must be readable and well-structured
- Sections must be clearly separated
- Code snippets are allowed but optional

25.4 Penalties Related to Report

- Missing report: **-100**
- Incomplete report: **-30**
- Report not explaining synchronization or termination logic: **-30**

Rules

1. Your program must be implemented in **C** for a POSIX-compatible system and must compile and run on Debian 11 (64-bit) in VirtualBox.
2. Your solution must use **multiple processes**. A solution that avoids process creation and simulates concurrency in a single process will not be considered a valid submission.
3. All processes must be created by the parent process. Child processes must not create additional system-level processes.
4. The system must initialize multiple word-carrier and letter-carrier processes per floor at startup.

5. Your implementation must include the following active subsystems:
 - a. word-carrier processes,
 - b. letter-carrier processes,
 - c. sorting processes,
 - d. one delivery elevator process,
 - e. one reposition elevator process,
 - f. one parent/coordinator process.
6. The input file must be scanned in **round-robin** order by the word-carrier subsystem. Word selection must be synchronized so that the same word is not claimed by multiple processes.
7. A word may be admitted into the system only if **both**:
 - a. the chosen arrival floor has capacity, and
 - b. the word's sorting floor has capacity.
8. Admission must be **all-or-nothing**. If both capacities cannot be reserved successfully, the word must be released and may be selected again later.
9. Each character task must carry enough metadata to allow correct reconstruction, including at least:
 - a. `word_id`,
 - b. the character itself,
 - c. original index,
 - d. source floor,
 - e. destination/sorting floor.
10. Each character task must be claimed by **exactly one** letter-carrier process and delivered **exactly once**.
11. Characters transported to a sorting floor must first be placed into the word's **sorting area** at the first suitable non-fixed empty slot.
12. Sorting processes must use the original word and the fixed/index information to reconstruct the word gradually.
13. If a sorting process encounters a character already in its correct final position, it must leave it there and mark that index as fixed.
14. If a character must be moved to a target position that is occupied by another non-fixed character, only a **swap** operation between those two positions is allowed.
15. Fixed positions are immutable: they can not be overwritten, they can not be swapped, newly arriving characters can not be placed there.
16. A floor may have multiple sorting processes, but **only one sorting process may operate on a specific word at a time**.
17. After delivering a character, a letter-carrier process must continue from its current floor. It must not automatically return to its previous floor.
18. If a letter-carrier become idle, it should issue a reposition request and use the reposition elevator to move to another floor.

19. The delivery elevator and reposition elevator must have **separate request queues** and must operate independently.
20. All system calls and synchronization calls must be checked for failure. All failures must be reported to `stderr` with informative error messages.
21. All shared-memory access that can be modified concurrently must be synchronized correctly.
22. Busy waiting is forbidden unless explicitly stated otherwise in the assignment. Polling loops that waste CPU time will be penalized.
23. Your program must terminate only when the entire system is complete, meaning: no unprocessed words remain, no active incomplete words remain.
24. On `Ctrl+C`, the program must terminate cleanly, release all system resources, and avoid leaving zombie processes behind.

Grading

Code — 60 Points

Component	Description	Points
Word Admission & Round-Robin	Correct round-robin scanning, atomic word claiming, correct all-or-nothing floor-capacity admission	10
Character Task Management	Correct generation, claiming, and one-time delivery of character tasks	10
Delivery Elevator	Correct delivery request handling and character transport	10
Reposition Elevator	Correct handling of idle-carrier relocation requests	5

Sorting Logic	Correct use of sorting area, fixed/index logic, move/swap behavior, and word completion detection	15
Synchronization Correctness	Correct use of shared memory synchronization, no unsafe concurrent modification, no duplicate claim races	5
Final Output File Correctness	Producing the file after system completion	5

Demo — 40 Points

Each student will attend an individual demo. The TA may ask the student to:

- explain the overall architecture,
- show how words are admitted,
- explain how a character is claimed and delivered,
- explain how sorting works,
- explain how fixed positions are protected,
- explain how the program avoids deadlocks, starvation, and duplicate claims,
- explain termination detection.

A student who cannot explain their own design and code may receive a very low demo score regardless of code quality.

A demo (~5 min) will be conducted for every student. The demo is graded independently from the code.

Penalties

Condition	Penalty
Code does not compile (make fails)	-100

No real multi-process design / process creation not meaningfully used	-50
Same word claimed by multiple processes due to incorrect synchronization	-30
Same character task delivered more than once	-30
Concurrent sorting on the same word	-30
Floor capacity check not atomic / inconsistent admission state	-30
Busy waiting / spin-style polling where blocking synchronization was expected	-40
Deadlock of any kind	-50
Starvation caused by obviously broken scheduling or request handling	-20
Memory leak (checked with valgrind or equivalent tools)	-30
Zombie processes present after normal completion or Ctrl+C	-30
Missing or broken Makefile	-20
Each -Wall compiler warning (first 10)	-2 each
More than 10 -Wall warnings	-30
No final output file or incorrect format	-30
Output file not sorted correctly	-15
No report submitted, or clearly insufficient report	-30

Binary executables included in submission	-10
Late submission	Not accepted

Notes on penalties

- If the program crashes under normal input, the submission cannot receive a passing implementation score.
- If fixed positions can be overwritten or swapped, the sorting logic is considered incorrect.
- If the program terminates while work is still pending, the termination logic is considered incorrect.

Submission Rules

- Place all source files, header files, your Makefile, and your report into a directory named with your student number.
- Compress that directory as a `.zip` archive.
- The archive must not include compiled binaries or object files.
- Your report must be submitted in **PDF** format.
- Your report must be written in **English**.
- Your Makefile must compile the program only; it must not execute it automatically.
- Any submission that significantly violates the submission format may be rejected without full content evaluation.
- The TAs will compile your code on their own systems.
- Identical or suspiciously similar submissions will receive severe academic penalties according to course policy.

Hints

- Design the system architecture before writing code. This assignment is too complex to solve correctly by improvising.
- Draw the process roles and the shared-memory structures on paper first.

- Clearly separate:
 - word admission,
 - character transportation,
 - sorting,
 - elevator request handling,
 - termination detection.
- Use one clear ownership rule for every shared resource:
 - who may claim it,
 - who may modify it,
 - how it becomes available again.
- Protect each word's sorting state with a word-level synchronization primitive.
- Define a consistent lock acquisition order to avoid deadlocks.
- Never block on long waits while holding a lock that other processes need.
- Test duplicate-character words carefully, such as words with repeated letters.
- Test cases where:
 - preferred floors are full,
 - multiple words target the same sorting floor,
 - multiple letter-carriers deliver to the same word concurrently,
 - multiple sorters exist on the same floor,
 - no work remains and the system should terminate.
- Check zombie cleanup explicitly after Ctrl+C.
- Use -Wall from the beginning and keep warnings at zero.
- Add debug logging while developing, but make sure your final output still conforms to the assignment requirements.